

Review Report on “On computing and the complexity of computing higher-order U-statistics, exactly”

Summary and overall assessment.

This paper proposes efficient and exact methods for computing V-statistics and U-statistics using Einstein summation. It also provides a theoretical characterization of the time complexity under idealized assumptions, together with several examples illustrating applications of the proposed approach.

Overall, I find the topic relevant and potentially useful, especially given the growing use of U-statistics in modern statistical applications. However, my impression is that the paper currently reads primarily as an application and review of existing Einsum-based operations and time-complexity analyses in the context of computing statistical quantities. To broaden the paper’s practical impact, I encourage the authors to clarify the computational details and provide more intuitive examples for the abstract concepts introduced throughout the paper. Doing so would make the general procedure more accessible and would likely encourage broader use beyond the specific examples discussed.

In particular, several concepts are currently presented in a rather abstract way without sufficiently concrete examples immediately following the definitions. I believe that making the exposition easier to follow would substantially increase the paper’s impact.

Regarding the characterization of optimal time complexity, it would be helpful to clarify what new techniques, conclusions, or insights the paper contributes relative to the existing literature on Einsum operations and algorithmic time complexity. My current impression, based on the main text, is that many of the general concepts are already well established in the literature on algorithmic complexity, whereas the specific statistical examples may not have been explicitly discussed before. If the paper contains additional new insights, I encourage the authors to highlight them more clearly.

In summary, I think this paper would be most useful if it were written for statisticians who work with U-statistics but may not have prior background in the Einsum operator or tensor-based algorithms. A more straightforward and accessible explanation of the efficient computation strategy would make the paper more useful and impactful than a primarily theoretical discussion of optimal time complexity, especially when it is unclear how that optimal complexity is achieved in practice. My specific comments are detailed below.

Major comments.

1. In Section 3.2, the discussion of computational cost appears to focus mainly on Algorithm 2, namely the Einsum operation, while the cost of Algorithm 1 seems to be largely omitted.

Moreover, on Page 6, Definition 3 states that “*there is no time complexity to access an entry in T , i.e., to evaluate T on α .*” Is this intended as a theoretical assumption, or is it meant

to hold in practice? It would be helpful to explain why the complexity of querying entries of large tensors can be ignored.

2. On Page 7, Definitions 4–5 introduce the general Einsum notation and operation using an input tuple list $\mathcal{A}$ and an output tuple B . However, the role of the output tuple B is not immediately clear. In the main examples, it appears that only the case $B = \emptyset$ is used.
 - (a) I noticed that additional examples with $B \neq \emptyset$ are provided in Appendix A. It would be helpful to explicitly mention in the main text that Example 1 illustrates the case $B = \emptyset$, while additional examples with $B \neq \emptyset$ are given in Appendix A. At present, the main text does not clearly highlight the new information provided in the appendix.
 - (b) In Appendix A, the labels “Example 2” and “Example 3” conflict with examples in the main text. These could be renamed as Examples A.2 and A.3, or Examples S.2 and S.3, to avoid confusion.
3. On Page 14, Proposition 1 characterizes the time complexity in terms of $\text{tw}(G_{\mathcal{A}})$. However, it is still unclear to me what $\text{tw}(G_{\mathcal{A}})$ would be in simple examples. I suggest adding simple examples to illustrate this quantity and explain how it is computed from the definitions.
4. On Page 14, Remark 3 discusses the difficulty of finding an ordering corresponding to the treewidth in practice. After reading this remark, it is unclear what “real” time complexity one should expect in practice, as opposed to the optimal theoretical complexity. It would be useful to clarify how well existing packages approximate the optimal ordering and how much inefficiency might arise in practice.

The paper mentions two strategies, greedy search and exhaustive search. Which of these is preferable in terms of approximating the optimal ordering? Some guidance on their relative advantages and limitations would be helpful.

Remark 3 also mentions that Zhang et al. (2025) used dynamic programming. How does the implementation in the present paper differ from that of Zhang et al. (2025) in order to accelerate the computation? Is dynamic programming avoided, or is the computational gain primarily due to sparsification in Section 3.3? This distinction is currently unclear.

5. On Page 15, Definition 11 introduces a partition π . What exactly does the partition π do, and what does $Q \in \pi$ mean in this context? It would be helpful to include simple examples illustrating these definitions.

Similarly, on Page 16, the paper states that “ $\mathcal{A}_{\pi}$ simply identifies all the indices that happen to belong to the same subset of the partition π . The procedure is presented in Algorithm 3.” The general description is difficult to interpret without a concrete example. I suggest adding a small example that walks through the construction step by step.

On Pages 17–18, Remark 4 is also difficult to follow. In particular, it is unclear how to interpret equation (4) and how the stated conclusion follows. Additional explanation or examples would make this part much clearer.

6. On Page 19, Remark 5 states that “ N_l and M can in principle be computed for concrete examples.” It would be useful to explain how this computation is performed in practice, perhaps through a simple example or a brief algorithmic description.

7. On Page 21, the last two lines state: “Then obviously $h_j^{(m)}$ allows for an MD structure, by letting $f_l(X_1, X_2) := A_1 \phi(Z_1)^\top A_2 \phi(Z_2)$ for $l < j$ and $f_j(X_1, X_2) := A_1 \phi(Z_1)^\top \phi(Z_2) Y_2$. The corresponding decomposition signature of $h_j^{(m)}$ is $((i, i+1))_{i=1}^{j-1}$.”

This explanation would be clearer if it were matched more explicitly to the notation introduced earlier. For example, what do $\mathcal{A} = \{A_1, \dots, A_k\}$ and $h_k(\cdot)$ in Definition 9 correspond to in this example?

On Page 22, the paper states that “the computational complexity up to order $m = 12$ is analyzed in Observation 2 in Appendix G.” Why is the analysis restricted to $m \leq 12$? What is the intuition behind the derivation of Observation 2? Is there an approximate upper bound, say $M(m)$, that could help readers understand how the problem complexity scales with m ? Some intuitive explanation would be helpful.

Minor comments.

8. On Page 8, Example 1 states: “The last example that we consider ...” What does “last example” refer to here? Is this a typo?
9. On Page 10, Definition 9, how should one interpret the double superscript notation “ $\in [m]^{**}$ ”?